

列表 (List)

列表的方法

功能	方法	作用	返回值	修改原列表
添加元素	<code>append(x)</code>	在末尾添加元素 x	无	是
	<code>insert(i, x)</code>	在索引 i 处插入元素 x	无	是
删除元素	<code>remove(x)</code>	删除第一个值为 x 的元素	无	是
	<code>pop()</code>	删除最后一个元素	被删除的元素	是
	<code>pop(i)</code>	删除索引 i 处的元素	被删除的元素	是
	<code>clear()</code>	清空所有元素	无	是
查找位置和频次	<code>index(x)</code>	查找 x 第一次出现的位置	索引	否
	<code>count(x)</code>	统计 x 出现的次数	次数	否
排序和反转	<code>sort()</code>	从小到大排序	无	是
	<code>sort(reverse=True)</code>	从大到小排序	无	是
	<code>reverse()</code>	反转元素顺序	无	是



列表 (List)

■ 与列表相关的其他操作

操作	类型	作用	修改原列表
<code>del 列表[i]</code>	关键字	删除索引 i 处的元素	是
<code>del 列表[start:end]</code>	关键字	删除切片范围内的元素	是
<code>len(列表)</code>	函数	返回元素个数	否
<code>sorted(列表)</code>	函数	返回排序后的新列表	否
<code>列表[::-1]</code>	切片	返回倒序后的新列表	否
<code>x in 列表</code>	运算符	判断 x 是否在列表中	否



列表 (List)

■ 列表的方法

- **方法**：属于某种数据类型的“专用函数”
- **调用语法**：列表名.方法名(参数)
- 列表的常用方法（按功能分为四组）：

功能	方法
添加元素	<code>append</code> 、 <code>insert</code>
删除元素	<code>remove</code> 、 <code>pop</code> 、 <code>clear</code>
查找位置和频次	<code>index</code> 、 <code>count</code>
排序和反转	<code>sort</code> 、 <code>reverse</code>



列表 (List)

■ 列表的方法：添加元素

方法	作用	返回值
列表.append(x)	在列表末尾添加元素 x	无
列表.insert(i, x)	在索引 i 的位置插入元素 x，原来该位置及之后的元素依次后移	无

```
languages = ['Python', 'Java', 'C++']
languages.append('JavaScript')    # 在列表末尾添加元素
print(languages)
languages.insert(1, 'SQL')        # 在索引 1 的位置插入元素
print(languages)
```

运行结果：

```
['Python', 'Java', 'C++', 'JavaScript']
['Python', 'SQL', 'Java', 'C++', 'JavaScript']
```



列表 (List)

■ 列表的方法：添加元素

```
nums = [1, 2]  
nums = nums.append(3)  
print(nums)
```



- nums 结果为 **None**
- **Python** 中 “没有返回值” 的方法实际返回的是 **None**
- append 已经修改了原列表，但返回值是 **None**，再赋值给 nums，列表就丢失了
- 正确写法是直接调用 `nums.append(3)`，不要赋值



列表 (List)

■ 列表的方法：删除元素

方法	作用	返回值
<code>列表.remove(x)</code>	删除指定元素 x（按值删除）	无
<code>列表.pop()</code>	删除最后一个元素	被删除的元素
<code>列表.pop(i)</code>	删除索引 i 的元素（按位置删除）	被删除的元素
<code>列表.clear()</code>	清空列表中的所有元素	无

```
languages = ['Python', 'SQL', 'Java', 'C++', 'JavaScript']
if 'Java' in languages:           # 先判断元素是否存在
    languages.remove('Java')      # 删除指定的元素
if 'Swift' in languages:         # 'Swift' 不存在，不执行删除
    languages.remove('Swift')
print(languages)
```

运行结果：['Python', 'SQL', 'C++', 'JavaScript']



列表 (List)

■ 列表的方法：删除元素

方法	作用	返回值
<code>列表.remove(x)</code>	删除指定元素 x（按值删除）	无
<code>列表.pop()</code>	删除最后一个元素	被删除的元素
<code>列表.pop(i)</code>	删除索引 i 的元素（按位置删除）	被删除的元素
<code>列表.clear()</code>	清空列表中的所有元素	无

```
nums = [10, 20, 30, 20]
```

```
nums.remove(20)          # 列表中有两个 20，只删除第一个（索引 1）
```

```
print(nums)              # 第二个 20 仍然保留
```

```
nums.remove(20)          # 再调用一次，才删除剩下的 20
```

```
print(nums)              当有重复元素时，remove 只删除第一个  
                           匹配的元素
```

```
运行结果：[10, 30, 20]
```

```
[10, 30]
```



列表 (List)

■ 列表的方法：删除元素

```
languages = ['Python', 'SQL', 'C++', 'JavaScript']  
last = languages.pop()           # 删除最后一个元素，并返回该元素  
print(last, languages)  
temp = languages.pop(1)          # 删除索引 1 的元素，并返回该元素  
print(temp, languages)  
languages.clear()                # 清空列表中的所有元素  
print(languages)
```

运行结果：

```
JavaScript ['Python', 'SQL', 'C++']  
SQL ['Python', 'C++']  
[]
```



列表 (List)

- 列表的方法：删除元素
 - 常见错误：

错误写法	报错信息	原因
<code>languages.remove('Swift')</code>	<code>ValueError: list.remove(x): x not in list</code>	要删除的元素不存在
<code>languages.pop(5)</code>	<code>IndexError: pop index out of range</code>	索引越界
<code>[] .pop()</code>	<code>IndexError: pop from empty list</code>	列表为空



列表 (List)

■ 列表的方法：删除元素

- remove 和 pop 的区别：

对比项	<code>remove(x)</code>	<code>pop(i)</code>
删除依据	按 值 删除	按 位置 删除
参数	要删除的元素，必须提供	要删除的索引，可省略，省略时删除最后一个元素
返回值	无 (None)	被删除的元素
有重复元素时	只删除第一个匹配的元素	只删除指定位置的元素
出错情况	元素不存在： <code>ValueError</code>	索引越界或列表为空： <code>IndexError</code>



列表 (List)

■ 使用 del 删除元素

- **del** 是 Python 的关键字，不是列表的方法，可以按位置删除元素，也可以删除整个变量

写法	作用
<code>del 列表[i]</code>	删除索引 i 的元素
<code>del 列表[start:end]</code>	删除切片范围内的元素
<code>del 列表</code>	删除整个变量，之后该变量不能再使用



列表 (List)

■ 使用 del 删除元素

- **del** 是 Python 的关键字，不是列表的方法，可以按位置删除元素，也可以删除整个变量

```
nums = [0, 1, 2, 3, 4, 5]
del nums[0]                # 删除索引 0 的元素
print(nums)
del nums[1:3]              # 删除索引 1~2 的元素
print(nums)
del nums                   # 删除整个变量
print(nums)
```

运行结果：

```
[1, 2, 3, 4, 5]
[1, 4, 5]
Traceback (most recent call last):
  File "test.py", line 7, in <module>
    print(nums)
NameError: name 'nums' is not defined
```



列表 (List)

■ 按位置删除 del 与 pop 的区别：

对比项	<code>del 列表[i]</code>	<code>列表.pop(i)</code>
类型	关键字	列表的方法
返回被删除的元素	不返回	返回
能否删除一段元素	能, <code>del nums[1:3]</code>	不能, 一次只删一个

列表 (List)

- **思考题**：如何删除列表中**所有的** 'Python'？

```
languages = ['Python', 'Python', 'Java']
```

```
for x in languages:  
    if x == 'Python':  
        languages.remove(x)  
print(languages)
```



```
languages = ['Python', 'Python', 'Java']
```

```
while 'Python' in languages:  
    languages.remove('Python')  
print(languages)
```



这两种方式是否都有效返回 ['Java']?

列表 (List)

- **思考题**：如何删除列表中**所有的** 'Python'？

```
languages = ['Python', 'Python', 'Java']
```

```
for x in languages:
```

```
    if x == 'Python':
```

```
        languages.remove(x)
```

```
print(languages)
```

关键在于：**for** 循环内部有一个“位置计数器”，每轮自动加1，但它不知道列表被删除了元素

列表 (List)

■ 思考题：如何删除列表中所有的 'Python'？

轮次	计数器位置	当前列表	取到的元素	执行操作
第 1 轮	0	['Python', 'Python', 'Java']	'Python'	删除，后面元素左移一位
第 2 轮	1	['Python', 'Java']	'Java'	不删除
第 3 轮	2	['Python', 'Java']	位置 2 不存在	循环结束

第 1 轮删除后，原来位置 1 的 'Python' 移到了位置 0，而计数器已经走到位置 1，于是它被跳过了



列表 (List)

■ 思考题：如何删除列表中所有的 'Python' ？

```
languages = ['Python', 'Python', 'Java']  
while 'Python' in languages:  
    languages.remove('Python')  
print(languages)
```



while 没有位置计数器。每一轮都重新检查整个列表中还有没有 'Python'，有就删掉第一个，列表怎么变化都不影响判断

	for 循环	while 循环
下一轮取哪个元素	由计数器决定，每轮位置加 1	不按位置取，每轮重新判断条件
删除元素后	计数器不知道元素已左移，会跳过元素	下一轮重新检查整个列表，不受影响



列表 (List)

■ 列表的方法：查找位置和统计频次

方法	作用	返回值
<code>列表.index(x)</code>	查找元素 x 第一次出现 的位置	索引
<code>列表.index(x, start)</code>	从索引 start 开始查找元素 x	索引
<code>列表.count(x)</code>	统计元素 x 出现的 次数	次数

```
items = ['Python', 'Java', 'Java', 'C++', 'Kotlin', 'Python']
print(items.index('Python'))      # 查找 'Python' 第一次出现的位置
print(items.index('Python', 1))   # 从索引 1 开始查找 'Python'
print(items.count('Python'))      # 统计 'Python' 出现的次数
print(items.count('Kotlin'))      # 统计 'Kotlin' 出现的次数
print(items.count('Swift'))       # 不存在的元素, 次数为 0
```

运行结果：0 5 2 1 0



列表 (List)

■ 列表的方法：查找位置和统计频次

- 查找不存在的元素时，count 返回 0，而 index 会报错：

```
print(items.index('Swift'))
```

ValueError: 'Swift' is not in list

- 建议先用 in 判断：

```
if 'Swift' in items:  
    print(items.index('Swift'))  
else:  
    print('Swift 不在列表中')
```



列表 (List)

■ 列表的方法：排序和反转

方法	作用	返回值
<code>列表.sort()</code>	将元素按从小到大排序	无
<code>列表.sort(reverse=True)</code>	将元素按从大到小排序	无
<code>列表.reverse()</code>	将元素的顺序反转	无

```
items = ['Python', 'Java', 'C++', 'Kotlin', 'Swift']
items.sort()                                # 按从小到大排序（字符串按字母顺序）
print(items)
items.sort(reverse=True)                    # 按从大到小排序
print(items)
items = ['Python', 'Java', 'C++', 'Kotlin', 'Swift']
items.reverse()
print(items)
```

运行结果：

```
['C++', 'Java', 'Kotlin', 'Python', 'Swift']
['Swift', 'Python', 'Kotlin', 'Java', 'C++']
['Swift', 'Kotlin', 'C++', 'Java', 'Python']
```



列表 (List)

■ 列表的方法：排序和反转

方法	作用	返回值
<code>列表.sort()</code>	将元素按从小到大排序	无
<code>列表.sort(reverse=True)</code>	将元素按从大到小排序	无
<code>列表.reverse()</code>	将元素的顺序反转	无

```
items = ['Python', 'Java', 'C++', 'Kotlin', 'Swift']
items.sort()                                # 按从小到大排序（字符串按字母顺序）
print(items)
items.sort(reverse=True)                    # 按从大到小排序
print(items)
items = ['Python', 'Java', 'C++', 'Kotlin', 'Swift']
items.reverse()
print(items)
```

运行结果：

```
['C++', 'Java', 'Kotlin', 'Python', 'Swift']
['Swift', 'Python', 'Kotlin', 'Java', 'C++']
['Swift', 'Kotlin', 'C++', 'Java', 'Python']
```



列表 (List)

■ 列表的方法：排序和反转

- **sort** 方法与 **sorted** 函数的区别：

```
nums = [35, 12, 99, 68]
```

```
new_nums = sorted(nums)
```

```
print(nums)
```

```
print(new_nums)
```

sorted 函数：返回排序后的新列表

原列表不变

运行结果：

```
[35, 12, 99, 68]
```

```
[12, 35, 68, 99]
```



列表 (List)

- 列表的方法：排序和反转
 - **sort** 方法与 **sorted** 函数的区别：

对比项	列表.sort()	sorted(列表)
类型	列表的方法	函数
是否修改原列表	修改	不修改
返回值	无 (None)	排序后的新列表



列表 (List)

■ 列表的方法：排序和反转

- 列表元素**类型不一致**时的排序：排序需要元素之间两两比较大小，所以能否排序取决于元素**能否相互比较**
- **情况一**：不同类型的数值可以排序

```
nums = [3, 1.5, True, 2]
nums.sort()
print(nums)
```

- **int、float、bool 都是数值，可以比较**
- **True 作为数值参与比较时相当于 1**
- **运行结果：[True, 1.5, 2, 3]**

列表 (List)

■ 列表的方法：排序和反转

- 列表元素**类型不一致**时的排序：排序需要元素之间两两比较大小，所以能否排序取决于元素**能否相互比较**
- **情况二**：数值与字符串混合，排序报错

```
items = [35, 'Python', 12]
```

```
items.sort()
```

整数和字符串无法比较大小

报错：TypeError: '<' not supported
between instances of 'str' and 'int'



列表 (List)

■ 列表的方法：排序和反转

- **reverse** 方法与切片 `[::-1]` 的区别：

```
nums = [35, 12, 99, 68]
```

```
new_nums = nums[::-1]
```

```
print(nums)
```

```
print(new_nums)
```

切片：得到倒序的新列表，原列表不变

```
nums = [35, 12, 99, 68]
```

```
nums.reverse()
```

```
print(nums)
```

reverse 方法：直接修改原列表

运行结果：

```
[35, 12, 99, 68]
```

```
[68, 99, 12, 35]
```

```
[68, 99, 12, 35]
```



列表 (List)

■ 列表的方法：排序和反转

- **reverse** 方法与切片 **[::-1]** 的区别：

对比项	列表.reverse()	列表[::-1]
类型	列表的方法	切片运算
是否修改原列表	修改	不修改
返回值	无 (None)	倒序后的新列表



列表 (List)

■ 列表生成式

- **问题**：创建一个列表，元素为 1 ~ 30 中能被 3 或 5 整除的数

➤ **写法一**：for 循环 + append

```
items = []                                # 创建空列表
for i in range(1, 31):                   # 遍历 1~30
    if i % 3 == 0 or i % 5 == 0:         # 判断能否被 3 或 5 整除
        items.append(i)                 # 满足条件，添加到列表
print(items)
```

是否有更简单的写法？



列表 (List)

■ 列表生成式

- **问题**：创建一个列表，元素为 1 ~ 30 中能被 3 或 5 整除的数

➤ 写法二：列表生成式

语法：[元素 (或任意表达式) for 变量 in 序列 if 条件]

作用：用一行代码代替“创建空列表、循环、判断、添加”四个步骤

```
items = [i for i in range(1, 31) if i % 3 == 0 or i % 5 == 0]  
print(items)
```

运行结果（两种写法相同）：

[3, 5, 6, 9, 10, 12, 15, 18, 20, 21, 24, 25, 27, 30]



列表 (List)

■ 列表生成式

- **问题**：创建一个列表，元素为 1 ~ 30 中能被 3 或 5 整除的数

➤ 两种写法的对应关系：

```
items = []  
for i in range(1, 31):  
    if i % 3 == 0 or i % 5 == 0:  
        items.append(i)  
print(items)
```

```
# 创建空列表  
# 遍历 1~30  
# 判断能否被 3 或 5 整除  
# 满足条件，添加到列表
```

```
items = [i for i in range(1, 31) if i % 3 == 0 or i % 5 == 0]  
print(items)
```

⚠ 没有筛选条件时，可以省略 if 部分



列表 (List)

■ 嵌套列表

- **问题**：保存 5 个学生 3 门课程的成绩，该怎么办？

	语文	数学	英语
学生 1	95	83	92
学生 2	80	75	82
学生 3	92	97	90
学生 4	80	78	69
学生 5	65	66	89

- **思路**：**列表的元素也可以是列表**。每个学生的 3 门课程组成一个列表，5 个学生的列表再组成一个大列表



列表 (List)

■ 嵌套列表

- 访问和修改元素

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]
print(scores[0])           # 第 1 个学生的 3 门成绩
print(scores[0][1])        # 第 1 个学生的第 2 门成绩
print(scores[-1][-1])      # 最后一个学生的最后一门成绩
scores[1][2] = 100         # 修改第 2 个学生的第 3 门成绩
print(scores[1])
print(len(scores), len(scores[0]))
```

运行结果：

```
[95, 83, 92]
83
89
[80, 75, 100]
5
3
```

列表 (List)

■ 嵌套列表

- 访问和修改元素
 - 两层索引的含义：scores[i][j]

写法	含义	类比表格
<code>scores[i]</code>	第 i+1 个学生的成绩列表	第 i+1 行
<code>scores[i][j]</code>	第 i+1 个学生的第 j+1 门成绩	第 i+1 行第 j+1 列
<code>len(scores)</code>	学生人数	行数
<code>len(scores[0])</code>	课程门数	列数



列表 (List)

■ 嵌套列表

- 遍历嵌套列表
 - **问题**：把所有学生的所有成绩放到一个列表中
 - **写法一**：两层 for 循环 + append

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]
all_scores = []
for row in scores:                                # 外层循环：取出每个学生的成绩列表
    for score in row:                              # 内层循环：取出该学生的每门成绩
        all_scores.append(score)
print(all_scores)
```



列表 (List)

■ 嵌套列表

- 遍历嵌套列表
 - **问题**：把所有学生的所有成绩放到一个列表中
 - **写法二**：列表生成式

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]  
all_scores = [score for row in scores for score in row]  
print(all_scores)
```

运行结果（两种写法相同）：

[95, 83, 92, 80, 75, 82, 92, 97, 90, 80, 78, 69, 65, 66, 89]



列表 (List)

■ 嵌套列表

- 遍历嵌套列表
 - 两种写法的对应关系：

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]  
all_scores = []
```

```
for row in scores:
```

外层循环：取出每个学生的成绩列表

```
    for score in row:
```

内层循环：取出该学生的每门成绩

```
        all_scores.append(score)
```

```
print(all_scores)
```

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]  
all_scores = [score for row in scores for score in row]  
print(all_scores)
```



列表 (List)

■ 嵌套列表

- 遍历嵌套列表
 - 列表生成式上增加分支判断
 - **问题**：找出所有 90 分及以上的成绩

```
high = [score for row in scores for score in row if score >= 90]  
print(high)
```

运行结果：

[95, 92, 92, 97, 90]



列表 (List)

■ 嵌套列表

- 创建嵌套列表 (两层 for 循环 + append)

```
scores = []  
for i in range(5):  
    temp = []  
    for j in range(3):  
        score = int(input('请输入第' + str(i + 1) + '个学生第' + str(j + 1) + '门课的成绩:'))  
        temp.append(score)  
    scores.append(temp)  
print(scores)
```

5 个学生
保存一个学生的成绩
3 门课程
一个学生的成绩列表作为一个元素添加

运行结果 (依次输入 15 个成绩后):

请输入第1个学生第1门课的成绩: 95

请输入第1个学生第2门课的成绩: 83

.....

请输入第5个学生第3门课的成绩: 89

[[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]



列表 (List)

■ 嵌套列表

- 创建嵌套列表 (嵌套列表生成式)

```
import random
```

```
scores = [[random.randint(60, 100) for j in range(3)] for i in range(5)]  
print(scores)
```

红色：内层；绿色：外层

- **内层** `[random.randint(60, 100) for j in range(3)]`：生成一个学生的 3 门成绩，得到一个小列表
- **外层** `[内层 for i in range(5)]`：把上面的小列表生成 5 次，组成大列表

列表 (List)

■ 嵌套列表

- 创建嵌套列表

-  不要用 * 创建嵌套列表

```
counters = [0] * 6
```

```
print(counters)
```

[0] * 6 可以快速创建包含 6 个 0 的列表 [0, 0, 0, 0, 0, 0]

```
scores1 = [[0] * 3] * 5
```

```
print(scores1) [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
scores1[0][0] = 95
```

```
print(scores1)
```

输出是 [[95, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]] 吗？

列表 (List)

■ 嵌套列表

- 创建嵌套列表

-  不要用 * 创建嵌套列表

```
scores1 = [[0] * 3] * 5
```

```
print(scores1) [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
scores1[0][0] = 95
```

```
print(scores1)
```

输出是 `[[95, 0, 0], [95, 0, 0], [95, 0, 0], [95, 0, 0], [95, 0, 0]]`
而非 `[[95, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]`

- 原因是外层的 * 5 没有创建五个独立的小列表，而是让这五行**共用同一个小列表**

列表 (List)

■ 嵌套列表

- 创建嵌套列表

-  不要用 * 创建嵌套列表

```
scores1 = [[0] * 3] * 5
```

```
print(scores1) [[0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]
```

```
scores1[0][0] = 95
```

```
print(scores1)
```

输出是 `[[95, 0, 0], [95, 0, 0], [95, 0, 0], [95, 0, 0], [95, 0, 0]]`
而非 `[[95, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]`

- 原因是外层的 * 5 没有创建五个独立的小列表，而是让这五行**共用同一个小列表**

列表 (List)

■ 嵌套列表

- 创建嵌套列表
 - ⚠ 不要用 * 创建嵌套列表

➤ 正确写法：

```
scores2 = [[0] * 3 for i in range(5)]  
print(scores2)           每个学生一个独立的成绩列表  
scores2[0][0] = 95  
print(scores2)
```

输出是 [[95, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0], [0, 0, 0]]



列表 (List)

■ 赋值与引用

- `[[0] * 3] * 5` 为什么修改一行，五行都会变？
 - 把列表赋值给另一个变量

```
nums1 = [35, 12, 99]
nums2 = nums1                      # 赋值: nums2 和 nums1 指向同一个列表
nums1[0] = 0                       # 通过 nums1 修改列表
print(nums1)
print(nums2)                       # nums2 也跟着变了
```

运行结果：

```
[0, 12, 99]
[0, 12, 99]
```

- 原因是 `nums2 = nums1` 没有复制出一个新列表
- 只是让 `nums2` 也指向 `nums1` 所指向的那个列表
- 列表在内存中只有一份
- 类似于C语言指针，`nums2 = nums1` 就像把一个指针的值赋给另一个指针，复制的是地址，而不是数据本身

列表 (List)

■ 赋值与引用

- `[[0] * 3] * 5` 为什么修改一行，五行都会变？
 - `id()` 函数返回数据在内存中的唯一编号，编号相同说明是同一个数据

```
nums1 = [35, 12, 99]
nums2 = nums1
print(id(nums1), id(nums2))    # 编号相同：同一个列表
```

```
nums3 = [35, 12, 99]
nums4 = [35, 12, 99]
print(id(nums3), id(nums4))    # 编号不同：内容相同的两个列表
print(nums3 == nums4)          # 内容相同
```

列表 (List)

■ 赋值与引用

- `[[0] * 3] * 5` 为什么修改一行，五行都会变？

➤ 回到上述问题

```
scores1 = [[0] * 3] * 5          # 把同一个成绩列表的引用重复放入 5 次
print(id(scores1[0]), id(scores1[1]), id(scores1[4]))
# 第 1、2、5 个学生成绩列表的编号相同，说明 5 个学生共用同一个列表
```

```
scores2 = [[0] * 3 for i in range(5)]    # 每次循环都新建一个成绩列表，共新建 5 次
print(id(scores2[0]), id(scores2[1]), id(scores2[4]))
# 第 1、2、5 个学生成绩列表的编号各不相同，说明每个学生有自己独立的列表
```

运行结果（编号每次运行会不同）：

```
139883244606016 139883244606016 139883244606016
139883244916544 139883242485760 139883242602176
```

列表 (List)

■ 浅拷贝与深拷贝

- `nums2 = nums1` 不会复制出新列表，怎样才能真正复制一个列表？
 - 概念
 - **拷贝**：创建一个新列表，把原列表的元素复制过去，修改原列表时拷贝结果不受影响
 - **浅拷贝**：只复制最外面一层。写法有 `列表.copy()` 和 `列表[:]` 两种
 - **深拷贝**：把每一层都复制一份，拷贝结果与原列表完全独立。写法是 `copy.deepcopy(列表)` (需要 `import copy`)
 - **⚠ 对于一维列表**，浅拷贝和深拷贝效果相同；对于**嵌套列表**，二者有区别



列表 (List)

■ 浅拷贝与深拷贝

- `nums2 = nums1` 不会复制出新列表，怎样才能真正复制一个列表？
 - 一维列表的拷贝
 - 一维列表只有一层，用浅拷贝就能得到完全独立的新列表

```
nums1 = [35, 12, 99]
nums2 = nums1.copy()
nums3 = nums1[:]
nums1[0] = 0
print(nums1)
print(nums2)
print(nums3)
```

```
# copy 方法：创建一个新列表，复制 nums1 中的元素
# 切片也能得到新列表，效果与 copy() 相同
# 修改 nums1
```

运行结果：

```
[0, 12, 99]
[35, 12, 99]
[35, 12, 99]
```

nums2 不受影响
nums3 不受影响



列表 (List)

■ 浅拷贝与深拷贝

- `nums2 = nums1` 不会复制出新列表，怎样才能真正复制一个列表？
 - 嵌套列表的拷贝
 - 嵌套列表有两层，浅拷贝只复制外层，内层的成绩列表仍然是共用的；**深拷贝**则两层都复制

```
import copy
```

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90], [80, 78, 69], [65, 66, 89]]
```

```
scores_copy = scores.copy()
```

```
scores_deep = copy.deepcopy(scores)
```

```
scores[0][0] = 100
```

```
print(scores[0])
```

```
print(scores_copy[0])
```

```
print(scores_deep[0])
```

```
# 浅拷贝：只复制外层列表，5 个学生的成绩列表仍是共用的
```

```
# 深拷贝：外层列表和 5 个成绩列表全部复制一份
```

```
# 修改第 1 个学生的语文成绩
```

运行结果：

```
# 浅拷贝跟着变了 [100, 83, 92]
```

```
# 深拷贝不受影响 [100, 83, 92]
```

```
[95, 83, 92]
```

一维列表用浅拷贝即可；嵌套列表需要完全独立的副本时，使用深拷贝

列表 (List)

■ 练习题

第1题 写出下面代码的运行结果

```
python
```

```
nums = [3, 1, 4, 1, 5]
nums.append(9)
nums.remove(1)
print(nums)
print(nums[1:4])
print(nums[::-1])
print(nums.count(1), nums.index(5))
```

列表 (List)

■ 练习题

代码	执行后的列表	说明
<code>nums = [3, 1, 4, 1, 5]</code>	<code>[3, 1, 4, 1, 5]</code>	初始列表, 5 个元素
<code>nums.append(9)</code>	<code>[3, 1, 4, 1, 5, 9]</code>	在末尾添加 9
<code>nums.remove(1)</code>	<code>[3, 4, 1, 5, 9]</code>	删除第一个 1 (索引 1), 后面元素依次左移

索引	0	1	2	3	4
元素	3	4	1	5	9

```
print(nums): [3, 4, 1, 5, 9]
```

```
print(nums[1:4]): [4, 1, 5]
```

```
print(nums[::-1]): [9, 5, 1, 4, 3]
```

```
print(nums.count(1), nums.index(5)): 1 3
```



列表 (List)

■ 练习题

第2题 写出下面生成式的运行结果，并改写为 for 循环

python

```
result = [x * 10 for x in range(1, 11) if x % 3 == 0]
```

result = [30, 60, 90]

```
result = []
```

```
for x in range(1, 11):
```

```
    if x % 3 == 0:
```

```
        result.append(x * 10)
```

```
print(result)
```

对应生成式最外层的方括号

对应 for x in range(1, 11)

对应 if x % 3 == 0

对应开头的 x * 10

列表 (List)

■ 练习题

第3题 已知3个学生3门课程的成绩，用列表生成式完成两个任务

```
python
```

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90]]
```

1. 取出所有学生的语文成绩（每行第1个）。
2. 筛选出英语成绩（每行第3个）不低于90分的学生。

```
scores = [[95, 83, 92], [80, 75, 82], [92, 97, 90]]
```

```
chinese = [row[0] for row in scores]  
print(chinese)
```

```
print([i + 1 for i in range(len(scores)) if scores[i][2] >= 90])
```

输出 [1, 3]